

دانشگاه خيام

دانشکده فنی مهندسی

گروه مهندسی کامپیوتر

پایان نامه دوره

کارشناسی مهندسی نرم افزار کامپیوتر

طراحی بک اند فروشگاه لوازم آرایشی آوید

استاد راهنما :

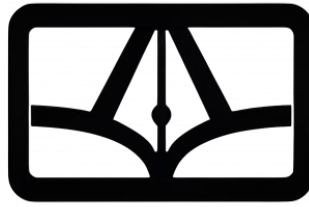
استاد علی رضائی

دانشجو :

محمد بیکی اکبرآباد

تابستان ۱۴۰۵

بسم الله الرحمن الرحيم



دانشگاه خيام

دانشکده فنی مهندسی

گروه مهندسی کامپیوتر

پایان نامه دوره

کارشناسی مهندسی نرم افزار کامپیوتر

طراحی بک اند فروشگاه لوازم آرایشی آوید

استاد راهنما :

استاد علی رضائی

دانشجو :

محمد بیکی اکبرآباد

تابستان ۱۴۰۵

سپاسگزاری :

در گام نخست، از خداوند متعال که در انجام این پژوهش مرا یاری فرمود، سپاسگزارم. بدین وسیله بر خود لازم می‌دانم تا مراتب سپاس و قدردانی خود را ابراز نمایم.

صمیمانه‌ترین تشکر خود را تقدیم استاد راهنمای ارجمندم، جناب آقای مهندس علی رضائی می‌نمایم. ایشان با راهنمودهای ارزشمند و نظارت دقیق، چراغ راه این پژوهش بودند. بدون دانش گسترده، صبر و دغدغه‌مندی‌های علمی ایشان، تحقق این پروژه با این سطح از کیفیت امکان‌پذیر نبود. بهره‌مندی من از محضر ایشان، افتخار بزرگی برای بنده بود و بی‌تردید در مسیر حرفه‌ای خویش همواره راهنمای من خواهد بود.

همچنین از تمامی اساتید محترم و مدیریت دانشگاه خیام که در طول دوران تحصیل با ارائه دانش و تجربیات خود، زمینه رشد علمی بنده را فراهم آوردند، تشکر می‌کنم.

از دوست و همکارم، سرکار خانم فاطمه کاظمی که در بخش طراحی رابط کاربری (فرانت‌اند) این پروژه با من همراه بود و با همفکری و تلاش خستگی‌ناپذیرش، به ارتقای کیفی و بصری این سیستم کمک شایانی نمود، نیز سپاسگزارم.

در پایان، از همه کسانی که هرچند به نحوی، با کلمه یا عمل، در پیشبرد این پژوهش یار و یاور بنده بوده‌اند، قدردانی می‌نمایم.

بااحترام،

محمد بیکی اکبرآباد

۲۰ شهریورماه ۱۴۰۵

تقدیم نامه:

این پایان نامه را به پیشگاه راهنمایان زندگی ام تقدیم می‌کنم؛ به پدر و مادرم که با ایثار و محبت خالصانه‌شان، بستر رشد و بالندگی مرا فراهم ساختند و به اساتید فرهیخته‌ام که با تزریق نور علم و خرد، مسیر دانایی را به من نمایاندند. امیدوارم این گام اندک در عرصه دانش، پاسخی شایسته به مهر و راهنمایی‌های ارزشمند آنان باشد.

چکیده

با گسترش تجارت الکترونیک، فروشگاه‌های آنلاین در مدیریت پویای موجودی، اعمال تخفیف‌های انعطاف‌پذیر و مدیریت نظرات با چالش مواجه‌اند. مسأله اصلی این پژوهش، طراحی سیستمی است که ضمن ارائه تجربه خرید روان، همگامی دقیق میان فروش و انبار را تضمین نماید. هدف، طراحی و پیاده‌سازی یک فروشگاه اینترنتی لوازم آرایشی به نام «آوید» با معماری همه‌جانبه (Full Stack) و جداسازی لایه‌های نرم‌افزاری است.

در روش‌شناسی، بک‌اند با فریم‌ورک جنگو (Django) و REST توسعه یافت. برای امنیت از احراز هویت JWT، برای بهینه‌سازی پایگاه داده از تکنیک‌های prefetch_related، برای ساخت خودکار پروفایل از سیگنال‌ها و برای مقاوم‌سازی API از مکانیزم Throttling استفاده شد. فرانت‌اند نیز با React.js پیاده‌سازی گردید.

نتایج این پژوهش، تولید یک سامانه پویاست که در آن همگام‌سازی لحظه‌ای سبد خرید با موجودی انبار، اعمال درصد تخفیف دلخواه، سیستم نظرات چندلایه (تایید مدیر و لایک/دیسلایک) و جستجوی زنده با موفقیت پیاده‌سازی شدند. این معماری باعث شد کدها به صورت مستقل توسعه یابند و قابلیت نگهداری سیستم افزایش یابد.

به عنوان پیشنهاد برای توسعه‌های آتی، اتصال درگاه‌های پرداخت واقعی، پیاده‌سازی اعلان‌های پیامکی خودکار، بهره‌گیری از کشینگ با Redis و استفاده از سیستم‌های توصیه‌گر مبتنی بر هوش مصنوعی پیشنهاد می‌گردد.

واژگان کلیدی: تجارت الکترونیک، جنگو، ری‌اکت، REST API، احراز هویت JWT، بهینه‌سازی پایگاه داده.

فهرست مطالب

عنوان	صفحه
فصل اول : تعریف مسئله	
۱-۱ مقدمه.....	۱۲
۲-۱ اهمیت و ضرورت موضوع.....	۱۳
۳-۱ اهداف پروژه.....	۱۵
۴-۱ تعریف واژه‌ها، مفاهیم و متغیرها.....	۱۶
فصل دوم : معرفی ابزار ها و تحلیل های پروژه	
۱-۲ مقدمه.....	۲۰
۲-۲ معرفی ابزار ها و تکنولوژی های استفاده شده	۲۱
۳-۲ روش تحلیل و طراحی	۲۲
۴-۲ مدل سازی و دیاگرام ها	۲۳
۱-۴-۲ دیاگرام کاربرد (usecase).....	۲۴
۲-۴-۲ دیاگرام کلاس (class).....	۲۵
۳-۴-۲ دیاگرام توالی (sequence).....	۲۶
فصل سوم : روش طراحی و پیاده سازی	
۱-۳ مقدمه	۲۹
۲-۳ فرایند انجام کار	۲۹
۳-۳ شرح طراحی و پیاده‌سازی هسته های اصلی بک اند	۳۰
۱-۳-۳ مدیریت پایگاه داده و معماری رویداد محور.....	۳۰
۲-۳-۳ امنیت ، احراز هویت و مقاوم سازی API.....	۳۱

۳-۳-۳ بهینه سازی کوئری ها و محاسبه منطق تجاری در سطح دیتا بیس..... ۳۲

۳-۳-۴ پیاده سازی جریان کاری ثبت سفارش ۳۴

فصل چهارم : نتیجه گیری ، بحث و پیشنهادها

۴-۱ مقدمه..... ۳۸

۴-۲ نتایج پروژه..... ۳۹

۴-۳ بحث بررسی ۴۰

۴-۴ پیشنهادات و توصیه ها برای ادامه کار در آینده..... ۴۱

منابع..... ۴۳

فصل اول:

تعریف مسئله

۱-۱. مقدمه

با تحول دیجیتال در دهه‌های اخیر، معماری نرم‌افزارهای مبتنی بر وب از سیستم‌های یکپارچه و سنگین (Monolithic) به سمت معماری‌های مقیاس‌پذیر و جداشده (Decoupled Architecture) حرکت کرده است. در این الگو، قلب تپنده و مغز متفکر هر پلتفرم تجارت الکترونیک، لایه بک‌اند (Backend) است که در سمت سرور اجرا می‌شود. این لایه وظیفه مدیریت پایدار داده‌ها، اعمال منطق تجاری (Business Logic)، تضمین امنیت ارتباطات و همگام‌سازی تراکشن‌ها با پایگاه داده را بر عهده دارد. یک بک‌اند ضعیف می‌تواند منجر به نشت اطلاعات کاربران، ناهماهنگی موجودی انبار با سفارشات و در نهایت فروپاشی کسب‌وکار شود.

این پژوهش به طراحی و پیاده‌سازی لایه بک‌اند یک سامانه فروشگاه اینترنتی لوازم آرایشی با نام تجاری «آوید» می‌پردازد. در این سامانه، از فریم‌ورک قدرتمند جنگو (Django) و جنگو REST Framework (DRF) برای پیاده‌سازی یک REST API استاندارد و مستقل استفاده شده است. بک‌اند این پروژه به عنوان یک سرویس‌دهنده (Server) عمل می‌کند که هیچ وابستگی‌ای به لایه نمایش (Frontend) ندارد و صرفاً درخواست‌های ورودی را پردازش کرده و داده‌های ساختاریافته (JSON) را برمی‌گرداند.

ابعاد این پروژه در لایه بک‌اند شامل محورهای زیر است:

۱. مدیریت داده‌ها و ORM: مدل‌سازی دقیق موجودیت‌های کسب‌وکار (محصولات، دسته‌بندی‌ها، سفارشات، آیتم‌های سفارش، نظرات و پروفایل کاربران) با استفاده از Object-Relational Mapping (ORM) جنگو برای ارتباط با پایگاه داده رابطه‌ای.

۲. لایه ارتباطی (API): ارائه سرویس‌های وب (Web Services) بر پایه پروتکل HTTP با فرمت استاندارد JSON، که امکان ارتباط هر کلاینتینی (وب‌سایت یا اپلیکیشن موبایل) را با سیستم فراهم می‌کند.

۳. امنیت و احراز هویت: پیاده‌سازی مکانیزم‌های امنیتی پیشرفته مبتنی بر توکن (JWT) برای کنترل دسترسی کاربران و محافظت از Endpoint ها.

۴. منطق تجاری (Business Logic): پیاده‌سازی محاسبات پیچیده سرور از جمله اعمال درصدهای تخفیف پویا برای هر محصول، همگام‌سازی لحظه‌ای موجودی انبار با سبد خرید، و مدیریت جریان کاری (Workflow) تایید نظرات کاربران.

۵. بهینه‌سازی و تاب‌آوری (Resilience): استفاده از تکنیک‌های کاهش ترافیک سرور (Throttling) جهت مقاوم‌سازی در برابر حملات سایبری، و بهینه‌سازی کوئری‌های پایگاه داده برای جلوگیری از افت پرفورمنس.

۱-۲. اهمیت و ضرورت موضوع

توسعه یک بک‌اند استاندارد برای یک پلتفرم تجارت الکترونیک، نیازی حیاتی است که فقدان آن منجر به ناپایداری سیستم و تهدید امنیت داده‌ها می‌شود. ضرورت‌های انجام این پروژه در لایه سمت سرور به شرح زیر است:

۱. جلوگیری از فروش کالای ناموجود (**Overselling**) و حفظ یکپارچگی تراکنش‌ها: در سیستم‌های غیربهمینه، عدم همگام‌سازی لحظه‌ای میان موجودی انبار و درخواست‌های خرید باعث ثبت سفارش برای کالاهای تمام‌شده می‌شود. این پروژه با پیاده‌سازی منطق اعتبارسنجی دقیق در لحظه ثبت سفارش (در سطح API) بررسی می‌کند که تعداد درخواستی کاربر از موجودی واقعی کالا در دیتابیس بیشتر نباشد و در صورت نامعتبر بودن، تراکنش را لغو کند.

۲. امنیت ارتباطات و حفاظت از داده‌های حساس: انتقال اطلاعات حساس مانند آدرس پستی مشتری، شماره تماس و تاریخچه خرید نیازمند لایه‌های امنیتی قوی است. استفاده از احراز هویت بی‌حالت (**Stateless**) مبتنی بر JWT و انتقال کلیدهای امنیتی سرور به متغیرهای محیطی (**.env**)، سیستم را در برابر حملات رایج وب مانند آسیب‌پذیری‌های (**OWASP**) مقاوم می‌سازد.

۳. مقاوم‌سازی سرور در برابر ترافیک مخرب و انبوه: API های عمومی فروشگاه‌های همواره در معرض حملات **Brute-force** یا ارسال درخواست‌های انبوه (**DDOS**) توسط ربات‌ها قرار دارند. پیاده‌سازی مکانیزم **Rate Limiting (Throttling)** برای محدودسازی تعداد درخواست‌های هر کلاینت در بازه‌های زمانی مشخص، ضرورت مقاوم‌سازی منابع سرور را ایجاب می‌کند.

مدیریت پویای قیمت‌گذاری: محاسبه قیمت‌های تخفیف‌خورده نباید در سمت کاربر (**Frontend**) انجام شود تا امکان دستکاری قیمت‌ها وجود نداشته باشد. این پروژه ضرورت محاسبه قیمت نهایی را بر اساس درصد‌های تخفیف متغیر، مستقیماً در سطح دیتابیس و با استفاده از **Annotate** ها پیاده‌سازی می‌کند.

۴. توسعه‌پذیری و همکاری تیمی (**Team Collaboration**): جداسازی کامل بک‌اند از فرانت‌اند امکان توسعه موازی تیم‌ها را فراهم می‌کند. بک‌اند طراحی شده در این پروژه به عنوان یک سرویس مستقل عمل

کرده و با ارائه مستندات دقیق API، بهره‌برداران و توسعه‌دهندگان فرانت‌اند را قادر می‌سازد بدون درگیر شدن با منطق دیتابیس، رابط کاربری را توسعه دهند.

۱-۳. اهداف پروژه

نقطه پایان این پروژه، ارائه یک API مستقل، امن و بهینه‌شده است که تمامی قوانین کسب‌وکار فروشگاه آوید را در سمت سرور مدیریت می‌کند و قابلیت اتصال به هر نوع کلاینت (وب یا موبایل) را داشته باشد.

هدف اصلی:

طراحی و پیاده‌سازی یک REST API مقیاس‌پذیر، امن و بهینه برای مدیریت فروشگاه اینترنتی آوید با استفاده از فریم‌ورک جنگو (Django) و جنگو. REST Framework (DRF).

اهداف جزئی:

۱. پیاده‌سازی سیستم احراز هویت بی‌حالت (Stateless) با استفاده از JSON Web Tokens (JWT) برای مدیریت نشست کاربران و کنترل دسترسی به سرویس‌های حساس مانند ثبت سفارش و مشاهده پروفایل.
۲. طراحی مدل داده‌ای نرمال‌شده و بهینه برای مدیریت موجودیت‌ها (محصولات، تخفیف‌ها، سفارشات، نظرات و علاقه‌مندی‌ها) با استفاده از قابلیت‌های ORM جنگو.
۳. بهینه‌سازی عملکرد پایگاه داده با استفاده از تکنیک‌های select_related و prefetch_related جهت رفع مشکل N+1 در استخراج داده‌های دارای روابط یک‌به‌چند (مانند نظرات یک محصول) و چندبه‌چند.

۴. پیاده‌سازی منطق تجاری پویا برای محاسبه قیمت نهایی محصول بر اساس درصدهای تخفیف متغیر، به صورت مستقیم در سطح دیتابیس با استفاده از `annotate` و `(ExpressionWrapper)` برای جلوگیری از دستکاری در سمت کاربر.

۵. مقاوم‌سازی سرویس‌های وب در برابر ترافیک انبوه با پیاده‌سازی `Throttling` برای محدود کردن نرخ درخواست کاربران مهمان `(Anonymous)` و احراز هویت شده `(Authenticated)`.

۶. اتوماسیون فرآیندهای دیتابیزی با استفاده از `(Django Signals)` مانند ساخت خودکار پروفایل کاربر بلافاصله پس از ثبت نام کاربر.

۷. مدیریت جریان کاری نظرات کاربران `(Workflow)` با اعمال فیلترهای تایید ادمین `(is_approved)`، شناسایی کاربران خریدار `(is_buyer)` و امکان ثبت پاسخ توسط مدیریت در سمت بک‌اند.

۸. حفظ یکپارچگی داده‌ها هنگام ثبت سفارش با کاهش همزمان موجودی کالاهای انبار `(Stock Management)` و ثبت تراکنش‌های مالی مرتبط با کدهای تخفیف.

۴-۱. تعریف واژه‌ها، مفاهیم و متغیرها

در این بخش، واژگان تخصصی و مفاهیم کلیدی مورد استفاده در این پژوهش به صورت علمی تعریف می‌شوند:

بک‌اند: `(Backend)` لایه‌ای از نرم‌افزار که در سمت سرور اجرا شده و مسئولیت پردازش منطق تجاری، تعامل با پایگاه داده، مدیریت صف `(Queue)` و پاسخگویی به درخواست‌های کلاینت را بر عهده دارد.

REST API (Representational State Transfer): یک سبک معماری برای طراحی سرویس‌های وب شبکه‌ای که در آن هر منبع (Resource) توسط یک URL مشخص شده و از متدهای استاندارد HTTP مانند GET برای خواندن، POST برای ایجاد، DELETE برای حذف (برای تبادل داده با فرمت JSON استفاده می‌شود).

ORM (Object-Relational Mapping): تکنیکی که به برنامه‌نویس اجازه می‌دهد بدون نوشتن کوئری‌های خام SQL، با استفاده از شی‌گرایی در زبان پایتون، جداول پایگاه داده را ایجاد و مدیریت کند که این امر امنیت و قابلیت حمل (Portability) سیستم را بالا می‌برد.

JWT (JSON Web Token): یک استاندارد باز (RFC 7519) برای انتقال امن اطلاعات (Claims) بین طرفین به صورت یک رشته JSON فشرده و رمزنگاری شده. در این پروژه JWT جایگزین نشست‌های مبتنی بر کوکی (Session) شده است.

N+1 (N+1 Query Problem): یک پدیده ناکارآمد در ORM ها که در آن، برای واکشی یک مجموعه از داده‌های اصلی (۱ کوئری)، به ازای هر رکورد برای واکشی داده‌های مرتبط آن N کوئری درخواست جدیدی به پایگاه داده ارسال می‌شود که منجر به افت شدید پرفورمنس سرور می‌گردد.

Throttling (محدودسازی نرخ): فرآیند کنترل میزان استفاده از منابع سرور توسط محدود کردن تعداد درخواست‌هایی که یک کاربر می‌تواند در یک بازه زمانی مشخص (مثلاً در یک دقیقه) به API ارسال کند، جهت جلوگیری از حملات سایبری.

Django Signals: مکانیزمی در فریم‌ورک جنگو که اجازه می‌دهد اکشن‌های خاصی در پاسخ به رویدادهای دیتابیس (مانند post_save پس از ذخیره یک رکورد) به صورت خودکار و بدون وابستگی مستقیم کدها اجرا شوند. (Event-Driven Architecture)

Stateless (بی‌حالت): ویژگی سرور که در آن هر درخواست HTTP از سمت کلاینت، تمام اطلاعات لازم برای پردازش را در خود دارد و سرور نیازی به حفظ نشست (Session) کاربر در حافظه خود ندارد.

موجودیت‌های دیتا (متغیرهای دیتابیس): در مدل داده این پروژه، جداول اصلی شامل (Product محصول)، (Category دسته‌بندی) (Order سفارش) (OrderItem آیتم سفارش که حاصل رابطه چند به چند میان سفارش و محصول است) (Review نظر کاربر) (Profile پروفایل کاربر) و (Coupon کد تخفیف) می‌باشند که به عنوان متغیرهای پایگاه داده در لایه ORM تعریف شده‌اند.

فصل دوم:

معرفی ابزار ها و تحلیل های پروژه

۲-۱. مقدمه

در مهندسی نرم افزار مدرن، طراحی یک سیستم تجارت الکترونیک مقیاس پذیر نیازمند گذار از معماری های یکپارچه (Monolithic) به سمت معماری های جداشده (Decoupled) است. در این الگو، لایه بک اند (Backend) به عنوان هسته مرکزی نرم افزار عمل کرده و هیچ وابستگی به رابط کاربری (Frontend) ندارد. این جداسازی باعث می شود که بک اند صرفاً به عنوان یک ارائه دهنده سرویس (Service Provider) عمل کرده و داده ها را از طریق پروتکل های استاندارد وب در اختیار کلاینت ها قرار دهد.

این فصل به معرفی ابزارها و تکنولوژی های به کار رفته در لایه سمت سرور پروژه فروشگاه «آوید» می پردازد. سپس روش تحلیل و طراحی سیستم بر اساس رویکرد شی گرا (OOD) تشریح شده و در نهایت، مدل سازی رفتار و ساختار سیستم با استفاده از دیگرام های استاندارد UML مستند می گردد. هدف از این مدل سازی، ایجاد یک درک مشترک از معماری سیستم، جریان داده ها و روابط موجودیت های پایگاه داده پیش از ورود به فاز پیاده سازی است.

۲-۲. معرفی ابزارها و تکنولوژی‌های استفاده شده

انتخاب صحیح ابزارها در پروژه‌های سمت سرور، تاثیر مستقیمی بر امنیت، پرفورمنس، توسعه‌پذیری و نگهداری سیستم دارد. در لایه بک‌اند این پروژه از مجموعه تکنولوژی‌های زیر استفاده شده است:

زبان پایتون (Python):

به عنوان زبان اصلی برنامه‌نویسی سمت سرور انتخاب شد. پایتون به دلیل سینتکس خوانا، کتابخانه‌های گسترده و پشتیبانی قوی از پارادایم شی‌گرایی (OOP)، انتخابی ایده‌آل برای توسعه سریع نرم‌افزار (RAD) است.

فریم‌ورک جنگو (Django):

یک فریم‌ورک وب سطح بالا مبتنی بر پایتون که از الگوی معماری (MVT (Model-View-Template پیروی می‌کند. جنگو به صورت پیش‌فرض دارای سیستم احراز هویت، مسیریابی URL، و لایه‌های محافظتی در برابر حملات رایج وب مانند CSRF، XSS و SQL Injectio است. طراحی ORM محور آن امکان مدیریت پایگاه داده را بدون نیاز به نوشتن کوئری‌های خام SQL فراهم می‌کند.

جنگو (DRF) REST Framework :

یک کتابخانه قدرتمند که بر روی جنگو ساخته شده و ابزارهایی برای ساخت آسان و امن REST API ها ارائه می‌دهد. این ابزار با ارائه Serialization تبدیل اشیاء پیچیده دیتابیس به فرمت JSON و بالعکس، سیستم Pagination (صفحه‌بندی) و Throttling (محدودسازی درخواست)، توسعه API ها را استاندارد می‌سازد.

پایگاه داده رابطه‌ای (RDBMS) :

در فاز توسعه از SQLite استفاده شد تا نیاز به تنظیمات سرور مجزا نباشد. ساختار ORM جنگو این امکان را فراهم کرده است که در فاز استقرار (Deployment)، با تغییر یک خط کد، پایگاه داده به سیستم‌های قدرتمندتری مانند PostgreSQL مهاجرت کند.

احراز هویت SimpleJWT :

برای پیاده‌سازی امنیت بی‌حالت (Stateless) از این کتابخانه استفاده شد. این تکنولوژی به جای ذخیره نشست (Session) در سرور، توکن‌های Access و Refresh را بر اساس استاندارد JSON Web Token صادر می‌کند که مقیاس‌پذیری سیستم را در صورت اتصال چندین سرور به شدت بالا می‌برد.

سیستم کنترل نسخه Git :

برای مدیریت کدهای منبع، ردیابی تغییرات و همکاری تیمی (توسعه همزمان بک‌اند و فرانت‌اند) از سیستم توزیعی Git و پلتفرم GitHub استفاده شد.

۲-۳. روش تحلیل و طراحی

روش تحلیل و طراحی این پروژه بر اساس روش مهندسی نرم‌افزار شی‌گرا (OOD) و با استفاده از زبان مدل‌سازی یکپارچه (UML 2.0) انجام شده است. در این رویکرد، سیستم به مجموعه‌ای از اشیاء (Objects) تجزیه می‌شود که هر کدام ویژگی‌ها (Attributes) و رفتارهای (Methods) خاص خود را دارند.

فرآیند طراحی بک‌اند در سه مرحله اصلی صورت گرفت:

۱. تحلیل نیازمندی‌ها: شناسایی موجودیت‌های داده‌ای (مانند کاربر، محصول، سفارش) و قوانین تجاری (مانند محاسبه پویای تخفیف، همگام‌سازی موجودی انبار و جریان کاری تایید نظرات).
۲. طراحی ساختار ایستا (Static Structure): تعیین روابط میان جداول دیتابیس با استفاده از قابلیت‌های ORM جنگو (One-to-One, One-to-Many, Many-to-Many) و طراحی دیاگرام کلاس.
۳. طراحی رفتار پویا (Dynamic Behavior): تعیین نحوه تعامل بازیگران (Actors) با سیستم و جریان پیام‌ها میان کلاینت، سرور و دیتابیس با استفاده از دیاگرام‌های کاربرد و توالی.

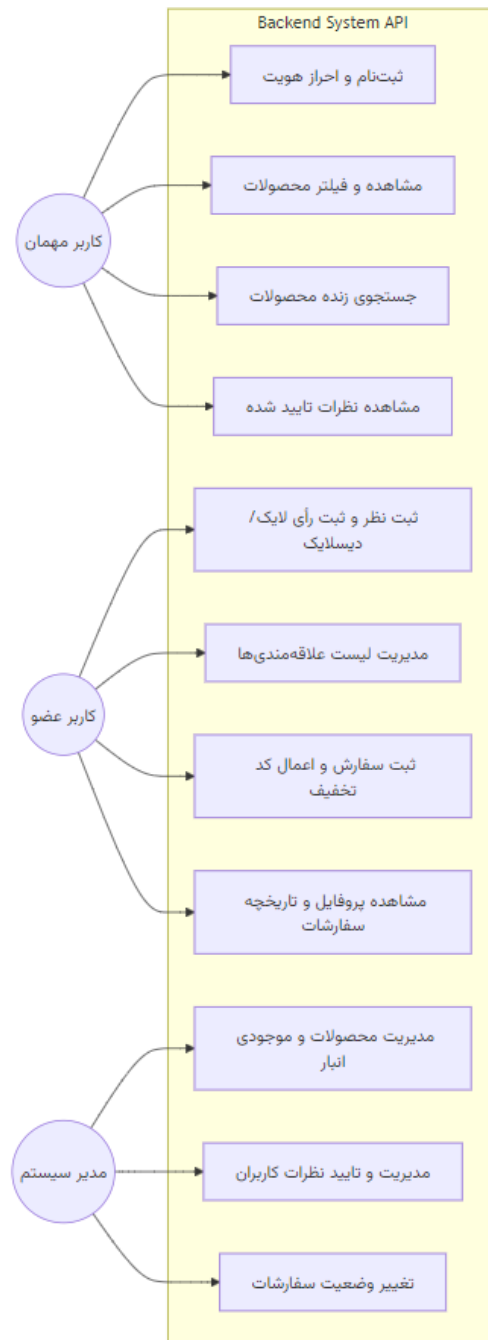
طراحی API ها نیز بر اساس اصول REST صورت گرفته است؛ به این معنی که هر URL یک منبع (Resource) را مشخص کرده و از متدهای استاندارد HTTP GET برای خواندن، POST برای ایجاد، PUT/PATCH برای به‌روزرسانی و DELETE برای حذف استفاده می‌کند.

۲-۴. مدل‌سازی و دیاگرام‌ها

در این بخش، ساختار سیستم بر اساس استانداردهای UML مدل‌سازی شده است.

۲-۴-۱. دیاگرام کاربرد (Use Case Diagram)

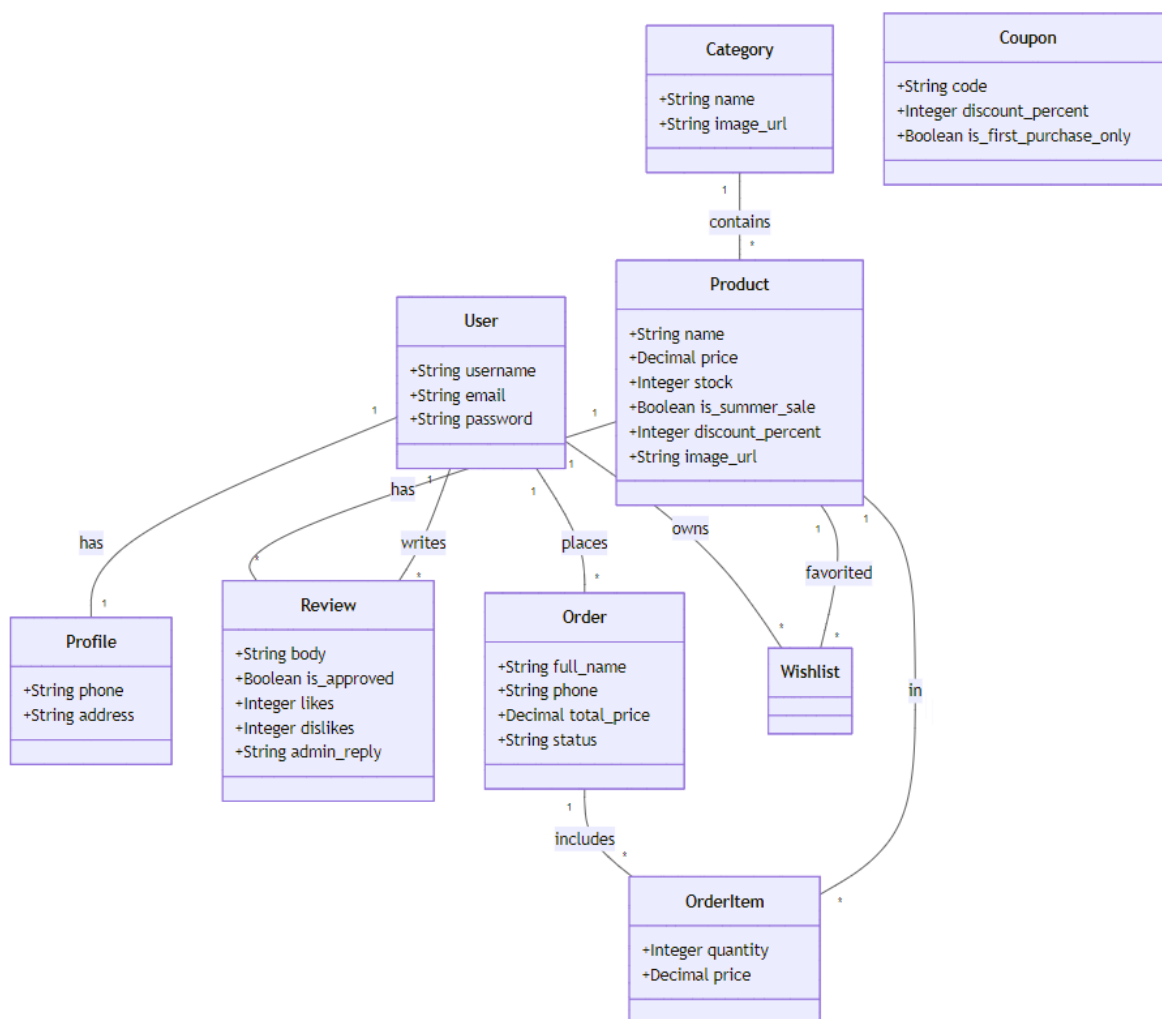
این دیاگرام تعامل بازیگران سیستم با قابلیت‌های (Use Cases) بک‌اند را نشان می‌دهد. بازیگران شامل کاربر مهمان، مهمان، کاربر احراز هویت شده (عضو) و مدیر سیستم هستند.



شکل ۲-۱: نمودار کاربرد

۲-۴-۲. دیاگرام کلاس (Class Diagram)

این دیاگرام ساختار داده‌ای و روابط بین موجودیت‌های سیستم را نمایش می‌دهد. هر کلاس به یک جدول در پایگاه داده (Model در جنگو) اشاره دارد.

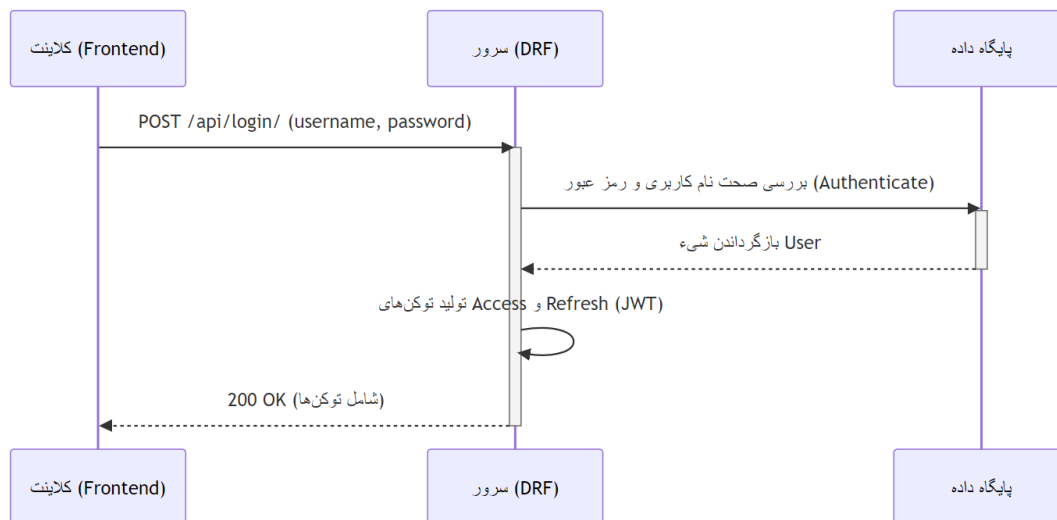


شکل ۲-۲: نمودار کلاس

۲-۴-۳. دیاگرام توالی (Sequence Diagram)

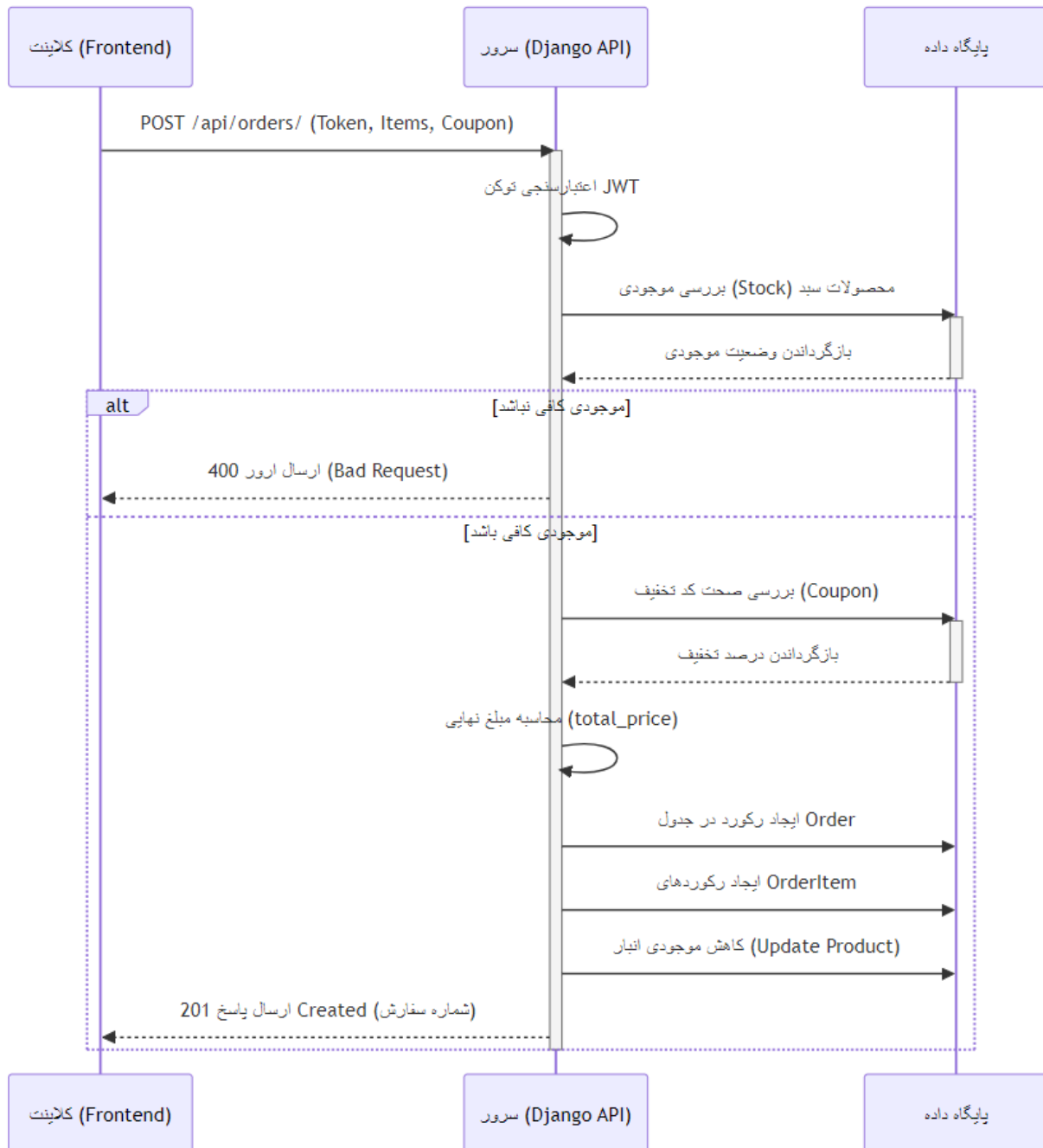
این دیاگرام‌ها توالی پیام‌های رد و بدل شده میان کلاینت (Frontend)، سرور (Django API) و پایگاه داده را در سناریوهای مهم نشان می‌دهد.

الف) سناریوی احراز هویت و ورود (Authentication):



شکل ۲-۳: نمودار توالی احراز هویت

ب) سناریوی ثبت نهایی سفارش (Checkout - دارای منطق تجاری پیچیده):



شکل ۲-۴: نمودار توالی ثبت سفارش

فصل سوم :

روش طراحی و پیاده سازی

۳-۱. مقدمه

پیاده‌سازی لایه بک‌اند (Backend) یک سیستم تجارت الکترونیک، نیازمند تبدیل نیازمندی‌های تجاری به کدهای قابل اجرا، امن و بهینه است. در این فصل، فرآیند انجام کار و نحوه پیاده‌سازی هسته‌های اصلی سمت سرور پروژه فروشگاه «آوید» با استفاده از فریم‌ورک جنگو (Django) و جنگو REST Framework (DRF) تشریح می‌شود. هدف از این فصل، نمایش جزئیات فنی پیاده‌سازی منطق تجاری، مدیریت پایگاه داده با ORM، و تامین امنیت ارتباطات است.

۳-۲. فرایند انجام کار

توسعه بک‌اند این پروژه بر اساس متدولوژی چابک (Agile) و به صورت زیر انجام شد:

۱. ساختاردهی پروژه: ایجاد اپلیکیشن products برای مدیریت موجودیت‌ها و تنظیمات اصلی پروژه

در settings.py

۲. مدل‌سازی داده‌ها: تعریف جداول پایگاه داده با استفاده از کلاس‌های پایتون (ORM) و اعمال

مهاجرت‌ها (Migrations) به پایگاه داده.

۳. توسعه API ها: نوشتن Serializer ها برای تبدیل داده‌ها و پیاده‌سازی View ها بر اساس متدهای

استاندارد HTTP.

۴. پیاده‌سازی امنیت: افزودن لایه احراز هویت JWT و مکانیزم‌های محافظت‌کننده (Throttling)

۵. بهینه‌سازی: بازیابی کوئری‌های دیتابیس برای رفع مشکلات پرفورمنس (مانند N+1) و استفاده از

محاسبات سطح دیتابیس.

۳-۳. شرح طراحی و پیاده‌سازی هسته‌های اصلی بک‌اند

در این بخش، چهار هسته اصلی سیستم که دارای بیشترین پیچیدگی مهندسی هستند، به همراه کدهای

پیاده‌سازی شده بررسی می‌شوند.

۳-۳-۱. مدیریت پایگاه داده و معماری رویدادمحور (Django Signals)

یکی از چالش‌های سیستم‌های کاربری، تاخیر در ایجاد جداول مرتبط (مانند پروفایل) است. برای حل

این مسئله، از معماری رویدادمحور استفاده شد. هنگامی که یک کاربر جدید در جدول User ثبت می‌شود،

یک سیگنال post_save به طور خودکار یک رکورد Profile برای او می‌سازد. این کار از ارورهای دیتابیس

(مثل Profile.DoesNotExist) در هنگام درخواست API جلوگیری می‌کند.

کد پیاده‌سازی سیگنال در فایل `signals.py`:

```
from django.db.models.signals import post_save
from django.contrib.auth.models import User
from django.dispatch import receiver
from .models import Profile
```

```
@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)
```

توضیح: دکوریتر `@receiver` تابع `create_profile` را به رویداد ذخیره مدل `User` متصل می‌کند.

اگر رکورد جدید ساخته شده بود (`if created`)، پروفایل مربوطه ایجاد می‌شود.

۲-۳-۳. امنیت، احراز هویت و مقاوم‌سازی API

برای امن‌سازی ارتباطات، از احراز هویت بی‌حالت (`Stateless`) مبتنی بر `JWT` استفاده شد. همچنین برای جلوگیری از حملات `Brute-force` و `DDOS`، مکانیزم `Rate Limiting` در تنظیمات سرور اعمال گردید تا تعداد درخواست‌های مجاز برای هر کاربر در دقیقه محدود شود.

کد پیاده‌سازی در فایل `settings.py`:

```

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle'
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '100/min', # محدودیت کاربر مهمان
        'user': '200/min' # محدودیت کاربر لاگین شده
    }
}

```

توضیح: در این پیکربندی، درخواست‌های کاربران مهمان به ۱۰۰ درخواست در دقیقه محدود شده‌اند. در صورت عبور از این محدودیت، سرور کد وضعیت ۴۲۹ Too Many Requests را برمی‌گرداند تا منابع پردازشی سرور حفظ شوند.

۳-۳-۳. بهینه‌سازی کوئری‌ها و محاسبه منطق تجاری در سطح دیتابیس

یکی از مهم‌ترین هسته‌های سیستم، API لیست محصولات است. در این بخش دو چالش مهندسی وجود داشت:

۱. محاسبه قیمت نهایی با درصدهای تخفیف متغیر برای هر محصول.

۲. مشکل N+1 در واكشی نظرات و دسته‌بندی‌ها.

برای حل این مسئله، از تکنیک `annotate` و ابزارهای `Case/When` در سطح دیتابیس استفاده شد تا محاسبات بدون انتقال داده‌ها به حافظه پایتون، مستقیماً توسط SQL انجام شوند.

کد پیاده‌سازی در فایل `views.py`:

```
from django.db.models import Case, When, F, FloatField, ExpressionWrapper, Value
```

فرمول محاسبه قیمت با درصد دلخواه: قیمت * (۱۰۰ - درصد تخفیف) / ۱۰۰

```
final_price_calc = ExpressionWrapper(
    F('price') * (Value(100.0) - F('discount_percent')) / Value(100.0),
    output_field=FloatField()
)
```

```
products = Product.objects.select_related('category').prefetch_related(
    Prefetch('reviews', queryset=Review.objects.filter(is_approved=True))
).annotate(
    final_price=Case(
        When(is_summer_sale=True, then=final_price_calc),
        default=F('price'),
        output_field=FloatField()
    )
)
```

توضیح:

`select_related` و `prefetch_related` با استفاده از `JOIN` در `SQL`، مشکل `N+1` را حل کرده و تعداد کوئری‌ها به دیتابیس را به حداقل می‌رسانند.

`ExpressionWrapper` فرمول ریاضی محاسبه تخفیف را به زبان `SQL` ترجمه می‌کند.

`Case/When`: شرط بررسی تخفیف را در خود جای داده است. با این روش، فیلتر کردن بر اساس قیمت نهایی (مثلاً جستجوی محصول بین ۱۰۰ تا ۲۰۰ هزار تومان) با دقت بالا روی قیمت‌های تخفیف‌خورده انجام می‌شود.

۳-۳-۴. پیاده‌سازی جریان کاری ثبت سفارش (Checkout & Stock Management)

ثبت سفارش حساس‌ترین بخش یک فروشگاه است. در این پروژه، `API` ثبت سفارش (`create_order`) طوری طراحی شد که یکپارچگی تراکنش‌ها (`Transaction Integrity`) را حفظ کند. این تابع ابتدا موجودی انبار را بررسی می‌کند، سپس کد تخفیف را اعتبارسنجی کرده و در نهایت در یک حلقه، موجودی محصولات را کاهش می‌دهد.

کد پیاده‌سازی در فایل `views.py`:

```
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def create_order(request):
    data = request.data
```

```
items = data.get('items', [])
```

```
coupon_code = data.get('coupon_code', None)
```

```
# بررسی موجودی انبار پیش از ثبت ۱۰
```

```
for item in items:
```

```
    product = Product.objects.get(id=item.get('id'))
```

```
    if product.stock < item.get('quantity'):
```

```
        return Response({"error": f"کافی نیست {product.name} موجودی محصول"},
```

```
status=status.HTTP_400_BAD_REQUEST)
```

```
# دریافت اطلاعات و محاسبه قیمت پایه ۲۰
```

```
profile, _ = Profile.objects.get_or_create(user=request.user)
```

```
phone = data.get('phone') or profile.phone
```

```
total = sum(float(item['price']) * item['quantity'] for item in items)
```

```
# اعمال کد تخفیف ۳۰
```

```
if coupon_code:
```

```
    try:
```

```
        coupon = Coupon.objects.get(code=coupon_code, is_active=True)
```

```
        if coupon.is_first_purchase_only and Order.objects.filter(user=request.user).exists():
```

```
            return Response({"error": "این کد فقط برای اولین خرید است"},
```

```
status=status.HTTP_400_BAD_REQUEST)
```

```
total = total - (total * (coupon.discount_percent / 100))
```

```
except Coupon.DoesNotExist:
```

```
pass
```

ثبت نهایی سفارش و آپدیت انبار ۴۰

```
order = Order.objects.create(user=request.user, phone=phone, total_price=total)
```

```
for item in items:
```

```
product = Product.objects.get(id=item.get('id'))
```

```
OrderItem.objects.create(order=order, product=product, quantity=item['quantity'])
```

```
product.stock -= item['quantity'] # کاهش موجودی
```

```
product.save()
```

```
return Response({"message": "سفارش ثبت شد", "order_id": order.id},
```

```
status=status.HTTP_201_CREATED)
```

استفاده از دکوریتر `@permission_classes([IsAuthenticated])` تضمین می‌کند که فقط کاربران دارای توکن معتبر می‌توانند سفارش ثبت کنند.

بررسی `coupon.is_first_purchase_only` منطق تجاری کدهای تخفیف انحصاری را پیاده‌سازی می‌کند.

حلقه نهایی کد، موجودی (`stock`) را در دیتابیس آپدیت می‌کند تا از فروش کالای بیش از حد موجودی (`Overselling`) جلوگیری شود.

فصل چهارم :

نتیجه گیری بحث و پیشنهاد ها

۴-۱. مقدمه

پس از طی کردن مراحل طراحی و پیاده‌سازی لایه بک‌اند (Backend) سامانه فروشگاهی «آوید»، ارزیابی دستاوردها و بررسی میزان موفقیت سیستم در برآورده کردن نیازمندی‌های تجاری و مهندسی، امری ضروری است. در این فصل، نتایج حاصل از پیاده‌سازی معماری جداشده (Decoupled Architecture) و راهکارهای مهندسی‌شده در سمت سرور جمع‌بندی می‌گردد. هدف از این ارزیابی، سنجش میزان موفقیت سیستم در ایجاد امنیت، پایداری و کارایی در پایگاه داده است و در نهایت، مسیرهای پیشنهادی برای توسعه این زیرساخت در آینده ارائه خواهد شد.

۴-۲. نتایج پروژه

با پیاده‌سازی این پروژه، یک REST API مستقل، امن و بهینه توسعه یافت که نتایج ملموس زیر را به همراه داشت:

ایجاد یک ساختار امن و بی‌حالت (Stateless): با موفقیت سیستم احراز هویت مبتنی بر JSON Web Token (JWT) پیاده‌سازی شد. این امر باعث شد تا سرور نیازی به ذخیره نشست‌ها (Sessions) در حافظه خود نداشته باشد و امکان مقیاس‌پذیری افقی (Horizontal Scaling) سرور در آینده فراهم گردید.

بهینه‌سازی چشمگیر پایگاه داده: با استفاده از تکنیک‌های select_related و prefetch_related در ORM جنگو، مشکل N+1 Query در زمان واکنشی محصولات و نظرات مرتبط با آن‌ها به طور کامل رفع شد. این امر باعث کاهش تعداد کوئری‌های ارسالی به پایگاه داده و در نتیجه کاهش زمان پاسخ‌دهی API (Response Time) شد.

مقاوم‌سازی API در برابر ترافیک مخرب: با اعمال مکانیزم Throttling (Rate Limiting)، سرور توانست درخواست‌های انبوه و ربات‌ها را مدیریت کرده و در صورت عبور از محدودیت تعیین‌شده، کد وضعیت ۴۲۹ Too Many Requests را بازگرداند که این امر از مصرف بی‌رویه منابع سرور جلوگیری می‌کند.

اتوماسیون فرآیندهای دیتابسی: با بهره‌گیری از Django Signals، فرآیند ایجاد پروفایل کاربر بلافاصله پس از ثبت‌نام به صورت خودکار انجام شد که این امر از بروز خطاهای یکپارچگی داده‌ها (Data Integrity) جلوگیری نمود.

یکپارچگی منطق تجاری: سیستم ثبت سفارش (Checkout) به گونه‌ای پیاده‌سازی شد که عملیات بررسی موجودی، محاسبه قیمت با تخفیف‌های پویا و ثبت نهایی سفارش در یک تراکنش امن انجام می‌شود و از فروش کالاهای ناموجود (Overselling) جلوگیری می‌گردد.

۴-۳. بحث و بررسی

تحلیل نتایج فوق نشان می‌دهد که انتخاب معماری REST API و جداسازی کامل بک‌اند از فرانت‌اند، تصمیمی کاملاً موفق بوده است. این جداسازی باعث شد تا توسعه نرم‌افزار به صورت تیمی با سرعت بالاتری انجام شود و وابستگی کدهای دیتابیس به کدهای رابط کاربری از بین برود.

از منظر امنیت، انتقال پردازش‌های حساس مانند محاسبه قیمت‌های نهایی و اعمال درصد‌های تخفیف به سطح پایگاه داده با استفاده از ExpressionWrapper و annotate، یکی از مهم‌ترین دستاوردهای این سیستم بود. این روش تضمین می‌کند که هیچ کاربری (حتی در صورت دستکاری درخواست‌ها در سمت کلاینت) نمی‌تواند قیمت محصولات را تغییر دهد، زیرا تمام محاسبات توسط موتور پایگاه داده انجام و بازگردانده می‌شوند.

همچنین، استفاده از متغیرهای محیطی (.env) برای نگهداری کلیدهای امنیتی جنگو (SECRET_KEY)، استاندارد بودن پروژه را برای محیط‌های استقرار (Deployment) نشان می‌دهد و ریسک نشت اطلاعات حساس در سیستم‌های کنترل نسخه مانند Git را به صفر رسانده است. سیستم مدیریت نظرات با اعمال فیلترهای تایید مدیر (is_approved) و قابلیت لایک/دیس‌لایک، توانست تعاملی پویا و در عین حال امن را فراهم آورد.

۴-۴. پیشنهادات و توصیه‌ها برای ادامه کار در آینده

اگرچه بک‌اند توسعه‌یافته در حال حاضر یک سیستم فروشگاهی کامل و استاندارد را پشتیبانی می‌کند، اما برای تبدیل شدن به یک پلتفرم تجارت الکترونیک در سطح سازمانی، انجام موارد زیر در آینده توصیه می‌گردد:

۱. اتصال درگاه‌های پرداخت بانکی واقعی: سیستم فعلی سفارشات را به صورت "پرداخت در محل" ثبت می‌کند. پیشنهاد می‌شود API های بک‌اند به درگاه‌های بانکی استاندارد متصل شوند و فرآیند Verify تراکنش در سمت سرور پیاده‌سازی گردد.

۲. استفاده از سیستم‌های کشینگ (Caching): برای کاهش بار پردازشی پایگاه داده در زمان بازدید کاربران متعدد، توصیه می‌شود از پایگاه داده‌های کلید-مقدار مانند Redis برای کش کردن داده‌های استاتیک مانند لیست محصولات و دسته‌بندی‌ها استفاده شود.

۳. پیاده‌سازی صف‌بندی وظایف (Task Queuing): برای عملیات‌هایی که زمان زیادی نیاز دارند (مانند ارسال پیامک وضعیت سفارش یا ایمیل فاکتور)، پیشنهاد می‌شود از ابزارهایی مانند Celery همراه با RabbitMQ استفاده شود تا این فرآیندها به صورت ناهمگام (Asynchronous) و در پس‌زمینه انجام گیرند و سرور اصلی مسدود نشود.

۴. مهاجرت به پایگاه داده قدرتمندتر: با توجه به اینکه در فاز توسعه از SQLite استفاده شده است، برای فاز تولید (Production) در فضای واقعی، توصیه می‌شود پایگاه داده به PostgreSQL مهاجرت کند تا همزمانی (Concurrency) بالاتری در تراکنش‌های مالی ارائه دهد.

۵. استقرار با استفاده از کانتینرها (Containerization): برای تسهیل فرآیند استقرار نرم افزار در سرورهای واقعی، بک اند و تمام نیازمندی های آن می توانند با استفاده از تکنولوژی Docker کانتینرسازی شوند تا محیط توسعه و محیط تولید کاملاً یکسان شوند و از خطاهای ناسازگاری محیطی جلوگیری گردد.

منابع

1. Django Software Foundation. (2024). *Django Official Documentation: Database access optimization*. Available at: <https://docs.djangoproject.com/en/5.0/topics/db/optimization/>
2. Django Software Foundation. (2024). *Django REST Framework Documentation*. Available at: <https://www.django-rest-framework.org/>
3. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine. Available at: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
4. Jones, M., Bradley, J., & Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force (IETF). Available at: <https://datatracker.ietf.org/doc/html/rfc7519>
5. Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.
6. OWASP Foundation. (2023). *REST Security Cheat Sheet*. The Open Web Application Security Project. Available at: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html
7. Vincent, W. S. (2022). *Django for Professionals: Production websites with Python & Django* (3rd ed.). Still River Press.